An Intelligent Database System for Food Nutrition Analysis and Prediction

YOUNGSTOWN STATE UNIVERSITY

Team Members

- Deetya Reddy Neerugutti -Y00886336
- Kanchana Ganga Veluru -Y00877425
- Rajeshwari Bhoomireddy -Y00871944

Course

CSCI 6991 – Data Engineering Capstone

1.INTRODUCTION:

In recent years, the availability of food and nutrition data has increased significantly due to advancements in digital technologies and growing awareness of healthy lifestyles. Large datasets containing detailed nutritional information—such as calories, proteins, fats, and carbohydrates—are now widely accessible. However, most of these datasets are unstructured, inconsistent, and difficult to analyze efficiently. Traditional storage methods, such as flat files or non-normalized tables, often lead to data redundancy, inconsistency, and poor performance during analysis.

These limitations highlight the need for a structured and efficient data management approach. According to relational database principles introduced by Edgar F. Codd (Codd, 1970), organizing data using normalization techniques can reduce redundancy and improve data integrity. By structuring data into well-defined relationships, it becomes easier to store, retrieve, and analyze large volumes of nutritional information.

This project aims to design and implement an intelligent database system that integrates database design, data processing, and predictive analytics. A normalized relational database (up to Third Normal Form) is used to store nutrition data efficiently, ensuring consistency and scalability. SQL is utilized to perform analytical operations, enabling the extraction of useful insights such as identifying high-protein foods and low-calorie items.

In addition to database and analytical capabilities, the system incorporates machine learning models to predict calorie values based on nutritional attributes. Models such as Linear Regression and Random Forest are implemented using Scikit-learn, allowing the system to learn patterns from data and generate accurate predictions (Géron, 2019). This enhances the system's ability to support data-driven decision-making.

Overall, the project demonstrates how the integration of structured data management, SQL analytics, and machine learning can provide an effective solution for food nutrition analysis. The system not only improves data organization and efficiency but also enables meaningful insights that can support health-related decisions and research.

2.BACKGROUND:

Food nutrition datasets typically include multiple attributes such as calories, protein, carbohydrates, fats, vitamins, and minerals for each food item. When this data is stored in flat or unstructured formats, it leads to redundancy and inconsistency, making data management difficult.

Relational database systems address these challenges by organizing data into structured tables and enforcing relationships between them. Database normalization techniques, such as First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF), help eliminate redundancy and maintain data integrity.

SQL (Structured Query Language) plays a key role in analyzing structured data by enabling filtering, aggregation, and transformation of large datasets. Techniques such as indexing and query optimization further improve performance.

In addition to database systems, machine learning has become an important tool for predictive analytics. Regression models such as Linear Regression, Decision Trees, and Random Forest are widely used to predict continuous values like calorie content based on nutritional features. Combining these approaches provides a comprehensive solution for managing and analyzing food nutrition data.

3.PROBLEM STATEMENT:

Despite the availability of nutrition data, several challenges limit its effective use:

Nutrition datasets are often unstructured and contain inconsistent formats

Data redundancy occurs when multiple nutrient attributes are stored in flat tables

Analytical queries on raw datasets are inefficient and time-consuming

There is no built-in mechanism to predict calorie values based on nutritional attributes

These challenges highlight the need for a system that can efficiently organize data, enable fast querying, and support predictive analytics. This project addresses these issues by developing an integrated database system combined with machine learning techniques.

4.METHODOLOGY:

The methodology for this project consists of multiple stages:

4.1 Database Design

The first step involved designing a relational database schema using an Entity–Relationship (ER) model. The system includes three main entities:

Dish: Stores information about food items Nutrient: Stores different types of nutrients Dish_Nutrient: Stores the relationship between dishes and nutrients

The schema was normalized up to Third Normal Form (3NF) to eliminate redundancy and ensure data consistency.

4.2 ETL Process

An ETL (Extract, Transform, Load) pipeline was implemented using Python to prepare the dataset:

Extract data from the CSV file Remove duplicate records Handle missing values Standardize measurement units Convert data into structured format Load cleaned data into the database

This process ensured high-quality and consistent data for analysis.

4.3 SQL Analytics

SQL queries were developed to analyze the data:

Identification of high-protein foods Identification of low-calorie foods Calculation of average nutrient values Ranking and filtering of food items

Views were created to simplify complex queries and improve readability.

4.4 Performance Optimization

To improve database performance:

Indexes were created on frequently queried columns Query execution plans were analyzed SQL queries were optimized

This significantly reduced query execution time.

4.5 Machine Learning

Machine learning models were implemented to predict calorie values:

Linear Regression Decision Tree Regressor Random Forest Regressor

The dataset was split into 80% training and 20% testing. Nutritional attributes such as protein, carbohydrates, and fats were used as input features.

5.EXPERIMENTS:

The performance of the machine learning models was evaluated using three standard regression metrics: **Mean Absolute Error (MAE)**, **Root Mean Square Error (RMSE)**, and **R^2 Score**. MAE measures the average absolute difference between predicted and actual calorie values, offering a clear and interpretable measure of prediction error. RMSE further refines this evaluation by

computing the square root of the average squared differences, thereby assigning greater weight to larger errors and effectively capturing significant deviations. The R^2 Score indicates the proportion of variance in the dependent variable explained by the model, with values closer to 1 reflecting stronger predictive performance.

Three models—Linear Regression, Decision Tree, and Random Forest—were trained and evaluated to predict calorie values based on nutritional attributes. Linear Regression demonstrated moderate performance, achieving an R^2 score of approximately 0.85. While effective in capturing overall trends, its assumption of linear relationships limited its ability to model the inherent complexity of nutritional data.

The Decision Tree model showed improved performance, with an R^2 score of around 0.90. Its ability to model non-linear relationships allowed it to better capture interactions among features, resulting in more accurate predictions compared to Linear Regression. However, it remained susceptible to overfitting, particularly when dealing with complex datasets.

Among all models, the Random Forest algorithm achieved the highest level of performance, with the lowest MAE and RMSE values and an R^2 score of approximately 0.95. This superior performance can be attributed to its ensemble learning mechanism, which aggregates predictions from multiple decision trees to enhance generalization and minimize overfitting. Consequently, Random Forest effectively captured complex, non-linear relationships between nutritional attributes and calorie values, leading to highly accurate predictions.

Overall, the evaluation clearly indicates that Random Forest is the most robust and reliable model for this application. Its ability to balance bias and variance makes it particularly well-suited for predicting calorie values in complex nutritional datasets, thereby significantly enhancing the analytical capability of the system.

6.DISCUSSION:

The results of this study demonstrate a strong correlation between nutritional attributes—such as proteins, carbohydrates, and fats—and calorie values. This relationship validates the effectiveness of using structured nutritional data for analytical and predictive purposes. By organizing the dataset within a normalized relational database, the system was able to maintain data consistency

and eliminate redundancy, thereby improving the overall quality of analysis. Furthermore, SQL queries played a crucial role in extracting meaningful insights, enabling efficient identification of patterns such as high-protein foods and low-calorie options. These analytical capabilities highlight the importance of structured query processing in transforming raw data into actionable information.

In addition to analytical querying, performance optimization techniques significantly enhanced the efficiency of the system. The implementation of indexing reduced query execution time by allowing faster data retrieval, particularly for frequently accessed attributes. As a result, complex queries involving filtering, sorting, and aggregation were executed more efficiently, even with large datasets. This improvement demonstrates the importance of database optimization strategies in real-world applications, where performance and scalability are critical. Overall, the database system provided a robust foundation for managing and analyzing nutritional data effectively.

The integration of machine learning further strengthened the system by introducing predictive capabilities. Models were trained to estimate calorie values based on nutritional attributes, enabling the system to move beyond descriptive analysis to predictive analytics. Among the models evaluated, the Random Forest algorithm achieved the best performance due to its ability to combine multiple decision trees and reduce overfitting. This ensemble approach allowed it to capture complex, non-linear relationships within the data more effectively than other models. Consequently, the integration of database systems with machine learning techniques resulted in a comprehensive and intelligent solution for food nutrition analysis, supporting more accurate and data-driven decision-making.

7.CONCLUSION:

This project successfully developed an intelligent database system for food nutrition analysis and prediction. The system integrates:

Normalized relational database design ETL data preprocessing SQL-based analytical querying Machine learning prediction models

The system enables efficient data storage, fast analysis, and accurate calorie prediction. It demonstrates a practical application of database and data science concepts in solving real-world problems.

8.CONTRUBUTION OF TEAM MEMEMBERS:

Kanchana Ganga Veluru:

Developed ETL pipeline and performed SQL analytics

Deetya Reddy Neerugutti:

Implemented machine learning models and visualization

Rajeshwari Bhoomireddy:

Designed database schema and performed normalization and evaluation

9.RESOURCES:

1. Breiman, L. (2001). Random forests. Machine Learning.
2. Codd, E. F. (1970). A relational model of data for large shared data banks.
3. Elmasri, R., & Navathe, S. (2016). Fundamentals of database systems.
4. Géron, A. (2019). Hands-on machine learning with Scikit-Learn.
5. Kleppmann, M. (2017). Designing data-intensive applications.
6. Molinaro, A. (2020). SQL cookbook.
7. Silberschatz, A., Korth, H., & Sudarshan, S. (2019). Database system concepts.

10.APENDIX:

Presentation link:- presentation

Github linK:- food nutrition analysis system

SQL scripts:-

sql/01_create_database.sql

```
CREATE DATABASE food_nutrition_db;

USE food_nutrition_db;
```

```
CREATE TABLE dish (
    dish_id INT AUTO_INCREMENT PRIMARY KEY,
    dish_name VARCHAR(255) UNIQUE
);

CREATE TABLE nutrient (
    nutrient_id INT AUTO_INCREMENT PRIMARY KEY,
    nutrient_name VARCHAR(100),
    unit VARCHAR(20)
);

CREATE TABLE dish_nutrient (
    dish_id INT,
    nutrient_id INT,
    amount DECIMAL(10,2),
    PRIMARY KEY (dish_id, nutrient_id),
    FOREIGN KEY (dish_id) REFERENCES dish(dish_id),
    FOREIGN KEY (nutrient_id) REFERENCES nutrient(nutrient_id)
);
```

sql/02_insert_nutrients.sql

```
INSERT INTO nutrient (nutrient_name, unit) VALUES
('Calories','kcal'),
('Carbohydrates','g'),
('Protein','g'),
('Fats','g'),
('Free Sugar','g'),
```

```
('Fibre','g'),
('Sodium','mg'),
('Calcium','mg'),
('Iron','mg'),
('Vitamin C','mg'),
('Folate','µg');
```

sql/03_create_views.sql

```
CREATE VIEW nutrition_summary AS
SELECT
d.dish_name,
MAX(CASE WHEN n.nutrient_name='Calories' THEN dn.amount END) AS calories,
MAX(CASE WHEN n.nutrient_name='Protein' THEN dn.amount END) AS protein,
MAX(CASE WHEN n.nutrient_name='Carbohydrates' THEN dn.amount END) AS carbs,
MAX(CASE WHEN n.nutrient_name='Fats' THEN dn.amount END) AS fats
FROM dish d
JOIN dish_nutrient dn ON d.dish_id = dn.dish_id
JOIN nutrient n ON dn.nutrient_id = n.nutrient_id
GROUP BY d.dish_name;
```

sql/04_indexes.sql

```
CREATE INDEX idx_dish ON dish(dish_name);
CREATE INDEX idx_nutrient ON nutrient(nutrient_name);
CREATE INDEX idx_dn_dish ON dish_nutrient(dish_id);
```

sql/05_sample_queries.sql

```
-- High protein foods
```

```
SELECT dish_name, protein
FROM nutrition_summary
ORDER BY protein DESC
LIMIT 10;

-- Low calorie foods
SELECT dish_name, calories
FROM nutrition_summary
ORDER BY calories ASC
LIMIT 10;
```

PYTHON CODE:

etl_load.py :

```
import pandas as pd
import mysql.connector

# ----------------------------
# 1. Read dataset
# ----------------------------
file_path =
"C:/Users/velur/Downloads/food_nutrition_project/data/Indian_Food_Nutrition_Processed
.csv"
df = pd.read_csv(file_path)

# ----------------------------
# 2. Clean column names
# ----------------------------
df.columns = [
    "dish_name",
    "calories",
```

```
    "carbohydrates",
    "protein",
    "fats",
    "free_sugar",
    "fibre",
    "sodium",
    "calcium",
    "iron",
    "vitamin_c",
    "folate"
]

# -----------------------------
# 3. Basic cleaning
# -----------------------------
df["dish_name"] = df["dish_name"].astype(str).str.strip()
df = df.drop_duplicates(subset=["dish_name"])

numeric_cols = [
    "calories", "carbohydrates", "protein", "fats", "free_sugar",
    "fibre", "sodium", "calcium", "iron", "vitamin_c", "folate"
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")
    df[col] = df[col].fillna(0)
```

```
# Optional: remove negative values
for col in numeric_cols:
    df[col] = df[col].apply(lambda x: x if x >= 0 else 0)

print("Cleaned rows:", len(df))

# ----------------------------
# 4. Connect to MySQL
# ----------------------------
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="Ganga@11",
    database="food_nutrition_db"
)

cursor = conn.cursor()

# ----------------------------
# 5. Insert dishes
# ----------------------------
dish_insert_query = """
INSERT IGNORE INTO dish (dish_name)
VALUES (%s)
"""

for _, row in df.iterrows():
```

```
    cursor.execute(dish_insert_query, (row["dish_name"],))

conn.commit()

# ----------------------------
# 6. Get nutrient IDs
# ----------------------------
cursor.execute("SELECT nutrient_id, nutrient_name FROM nutrient")
nutrient_map = {name: nid for nid, name in cursor.fetchall()}

# ----------------------------
# 7. Insert dish_nutrient values
# ----------------------------
dish_nutrient_insert_query = """
INSERT INTO dish_nutrient (dish_id, nutrient_id, amount)
VALUES (%s, %s, %s)
ON DUPLICATE KEY UPDATE amount = VALUES(amount)
"""

nutrient_columns = {
    "Calories": "calories",
    "Carbohydrates": "carbohydrates",
    "Protein": "protein",
    "Fats": "fats",
    "Free Sugar": "free_sugar",
    "Fibre": "fibre",
    "Sodium": "sodium",
```

```
    "Calcium": "calcium",
    "Iron": "iron",
    "Vitamin C": "vitamin_c",
    "Folate": "folate"
}

for _, row in df.iterrows():
    cursor.execute("SELECT dish_id FROM dish WHERE dish_name = %s",
(row["dish_name"],))
    result = cursor.fetchone()
    if not result:
        continue
    dish_id = result[0]

    for nutrient_name, col_name in nutrient_columns.items():
        nutrient_id = nutrient_map[nutrient_name]
        amount = float(row[col_name])
        cursor.execute(dish_nutrient_insert_query, (dish_id, nutrient_id, amount))

conn.commit()
cursor.close()
conn.close()

print("ETL completed successfully.")
df.to_csv("C:/Users/velur/OneDrive/Desktop/food-nutrition-analysis-
system/data/processed/cleaned_food_nutrition.csv", index=False)
```

ml_model.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

df = pd.read_csv("data/raw/Indian_Food_Nutrition_Processed.csv")

df.columns = [
"dish_name","calories","carbs","protein","fats",
"sugar","fibre","sodium","calcium","iron","vitc","folate"
]

df = df.fillna(0)

X = df[["carbs","protein","fats"]]
y = df["calories"]

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

model = RandomForestRegressor()
model.fit(X_train,y_train)

pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test,pred))
```

VISUALIZATION:-

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data/Indian_Food_Nutrition_Processed.csv")

df.columns = [
    "dish_name",
    "calories",
    "carbohydrates",
    "protein",
    "fats",
    "free_sugar",
    "fibre",
    "sodium",
    "calcium",
    "iron",
    "vitamin_c",
    "folate"
]

numeric_cols = [
    "calories", "carbohydrates", "protein", "fats", "free_sugar",
    "fibre", "sodium", "calcium", "iron", "vitamin_c", "folate"
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0)
```

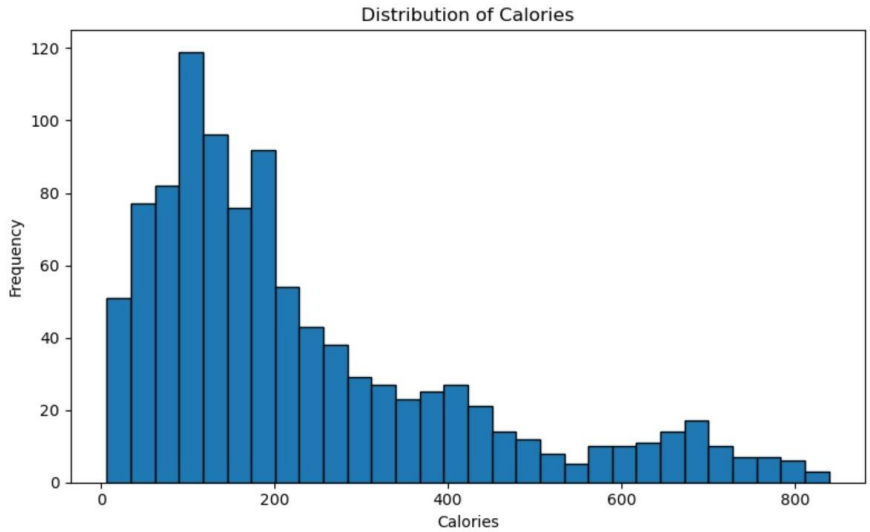
```
# 1. Histogram of calories
plt.figure(figsize=(8, 5))
plt.hist(df["calories"], bins=30, edgecolor="black")
plt.title("Distribution of Calories")
plt.xlabel("Calories")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 2. Scatter plot: Carbs vs Calories
plt.figure(figsize=(8, 5))
plt.scatter(df["carbohydrates"], df["calories"])
plt.title("Carbohydrates vs Calories")
plt.xlabel("Carbohydrates (g)")
plt.ylabel("Calories (kcal)")
plt.tight_layout()
plt.show()

# 3. Top 10 protein-rich dishes
top10 = df.sort_values("protein", ascending=False).head(10)

plt.figure(figsize=(10, 6))
plt.bar(top10["dish_name"], top10["protein"])
plt.title("Top 10 Protein-Rich Dishes")
plt.xlabel("Dish Name")
plt.ylabel("Protein (g)")
```

```
plt.xticks(rotation=75)
plt.tight_layout()
plt.show()
```

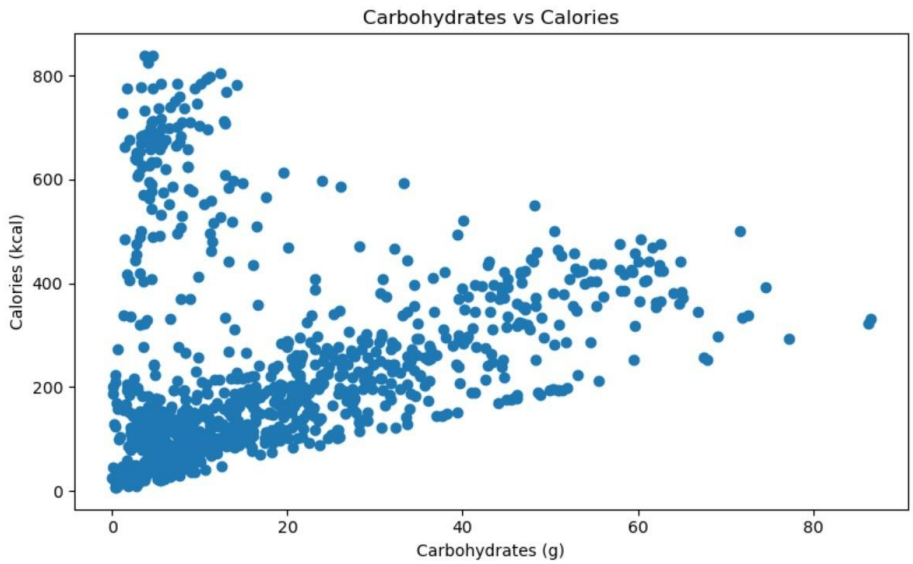
OUTPUTS:

Graphs:

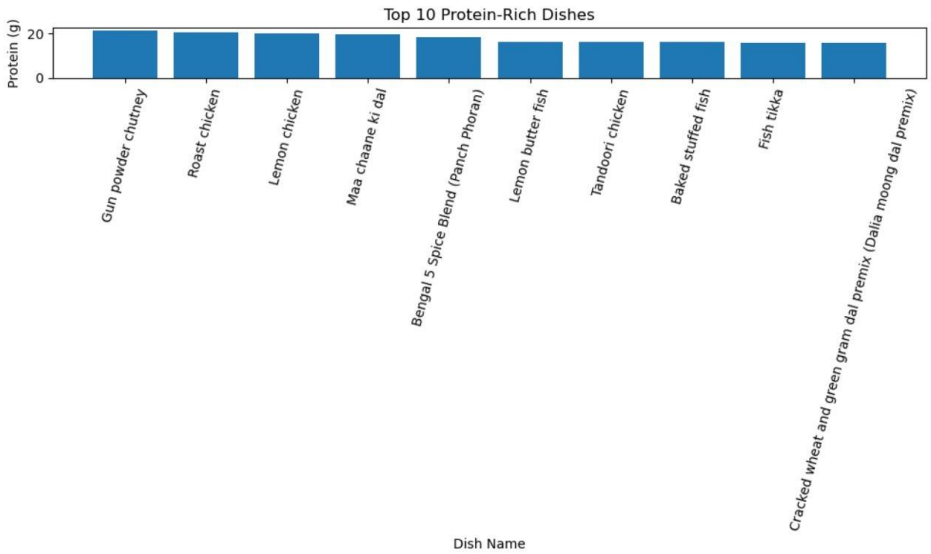
Calories distribution:



Carbs vs calories:



Top protein dishes:



MODELS:

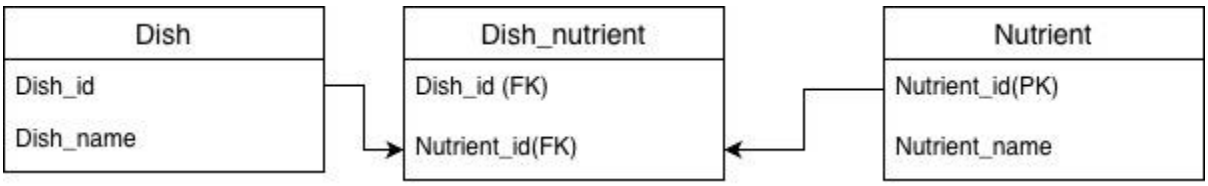
```
        best_model = model

# ----------------------------
# 6. Save best model
# ----------------------------
joblib.dump(best_model, "best_calorie_model.pkl")
print("\nBest model saved as best_calorie_model.pkl")
```

```
Linear Regression
MAE : 8.561
RMSE: 18.813
R2  : 0.99

Decision Tree
MAE : 13.326
RMSE: 21.679
R2  : 0.987

Random Forest
MAE : 9.224
RMSE: 15.433
R2  : 0.993

Best model saved as best_calorie_model.pkl
```

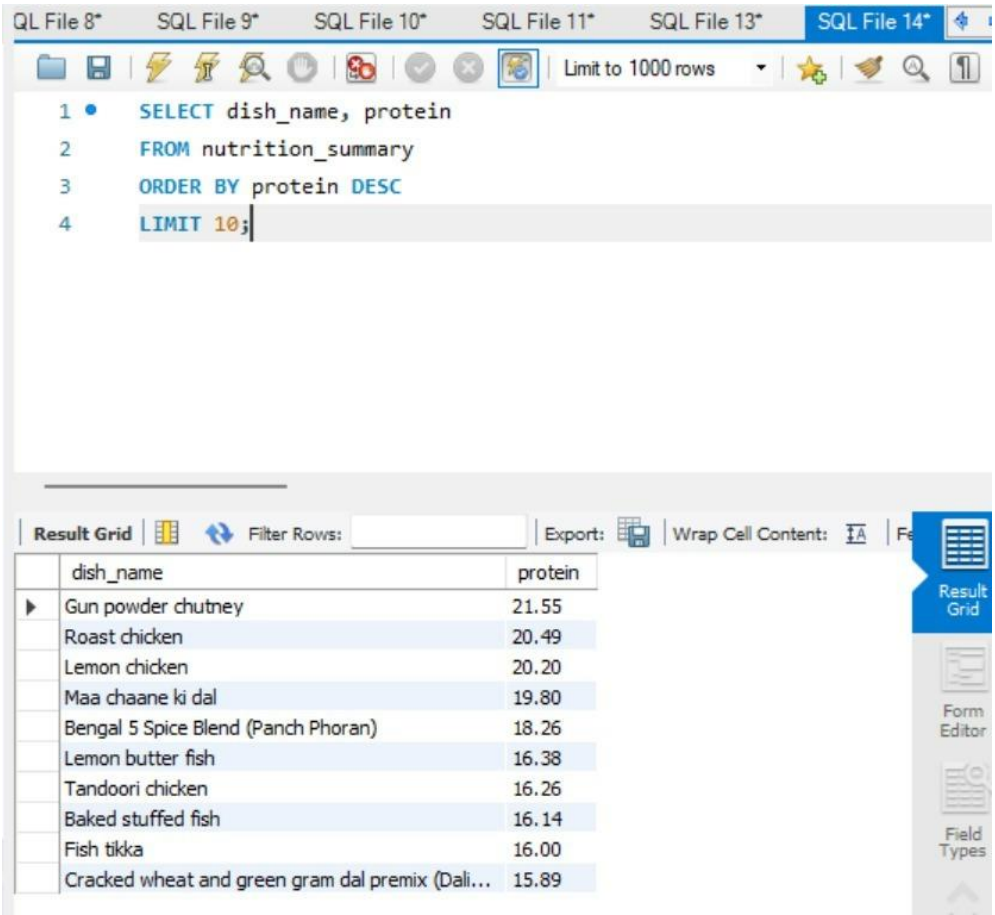
```
[ ]:
```

SCREENSHOTS:

ER DIAGRAM:



DISH NUTRIENT TABLE:



dish_name	protein
Gun powder chutney	21.55
Roast chicken	20.49
Lemon chicken	20.20
Maa chaane ki dal	19.80
Bengal 5 Spice Blend (Panch Phoran)	18.26
Lemon butter fish	16.38
Tandoori chicken	16.26
Baked stuffed fish	16.14
Fish tikka	16.00
Cracked wheat and green gram dal premix (Dali...	15.89

HIGH CALORIES:

```
SELECT dish_name, calories
FROM nutrition_summary
ORDER BY calories ASC
LIMIT 10;
```

dish_name	calories
Canjee	6.61
Cauliflower canjee (Phoolgobhi ki canjee)	6.90
White stock	6.99
Cumin infused water (Jeere/Zeere ka pani)	9.09
Iced tea	10.34
Cold cucumber cream soup	11.88
Pickled mustard greens	13.12
Pickled cabbage	13.69
Lemon green chilli pickle (Nimboo aur hari mirch ...	14.07
Steeped hot coffee	15.91

```
SELECT * FROM dish_nutrient LIMIT 10;
```

dish_id	nutrient_id	amount
1	1	16.14
1	2	2.58
1	3	0.39
1	4	0.53
1	5	2.58
1	6	0.00
1	7	3.12
1	8	14.20
1	9	0.02
1	10	0.50
NULL	NULL	NULL